

GUÍA PRÁCTICA #6

OPERACIONES: Creación de Mi Portafolio en React usando Componentes y Estados.

PASO #1: Instalar React (Revisar Guía Práctica #1 y #2).

Instalar React, las dependencias, realizar la limpieza correspondiente, instalar Bootstrap (Revisar Guía Práctica #1 y #2).

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs  
$ npm create vite@latest portafolio -- --template react
```

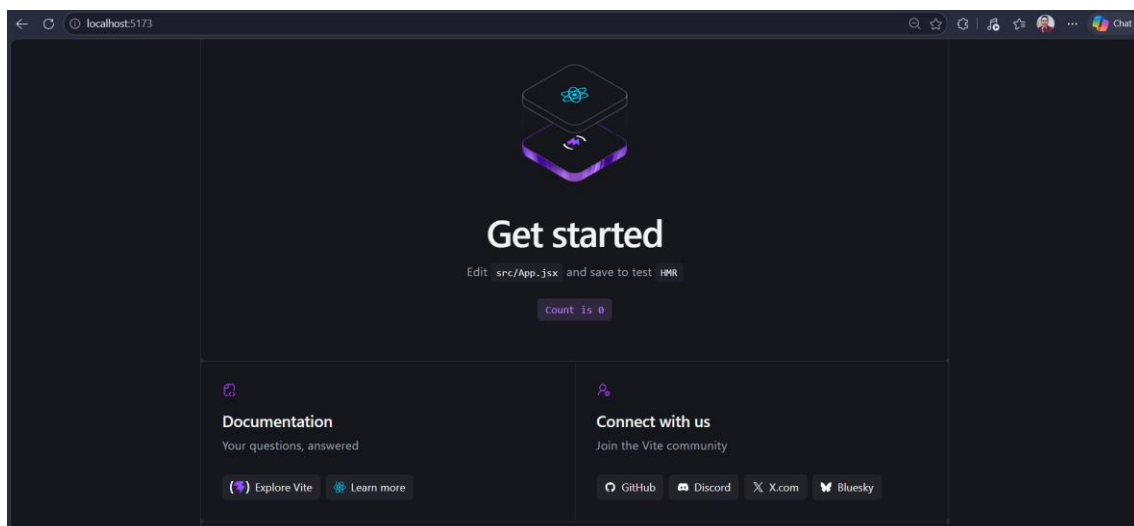
```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs  
$ cd portafolio
```

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/portafolio  
$ npm install
```

```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/portafolio  
$ npm run dev
```

```
VITE v8.0.14 ready in 399 ms
```

```
→ Local:   http://localhost:5173/  
→ Network: use --host to expose  
→ press h + enter to show help
```

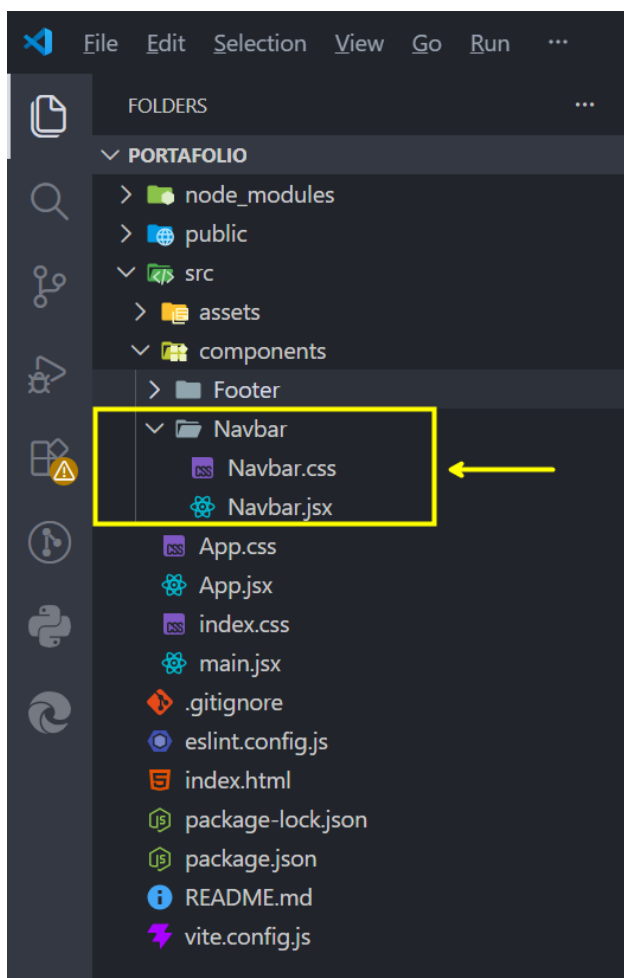


```
gbarb@LAPTOP-69814VSI MINGW64 /c/xampp/htdocs/portafolio  
$ npm install bootstrap
```

PASO 2: Estructura de Archivos.

Para mantener las buenas prácticas que ya hemos visto, vamos a encapsular tanto la lógica como los estilos de este componente.

En el explorador de VS Code, dentro de la carpeta **src/components**, creamos una carpeta de nombre **Navbar** con dos archivos nuevos:



PASO 2: El Componente React (Navbar.jsx).

Abre el archivo **Navbar.jsx**. Aquí utilizaremos `useState` para controlar si el menú móvil está abierto o cerrado. Cambiar "Giancarlos Barboza" por sus respectivos nombres.

```
import { useState } from 'react';
import './Navbar.css';

const Navbar = () => {
  // Estado para controlar el menú hamburguesa en móviles
  const [isOpen, setIsOpen] = useState(false);

  return (
    <nav className="navbar">
      { /* Sección del Logo */ }
      <div className="navbar-logo">
        <span className="text-cyan">Giancarlos</span> Barboza
      </div>

      { /* Enlaces de Navegación */ }
      <div className={`navbar-links ${isOpen ? "open" : ""}`}>
        { /* Al hacer clic en un enlace en móvil, cerramos el menú */ }
        <a href="#sobre-mi" onClick={() => setIsOpen(false)}>Perfil</a>
        <a href="#tech-stack" onClick={() => setIsOpen(false)}>Tech
Stack</a>
        <a href="#proyectos" onClick={() =>
setIsOpen(false)}>Proyectos</a>
        <a href="#servicios" onClick={() =>
setIsOpen(false)}>Servicios</a>
        <a href="#contacto" onClick={() => setIsOpen(false)}>Contacto</a>
      </div>

      { /* Ícono de Menú Hamburguesa (Solo visible en móviles) */ }
      <div className="navbar-toggle" onClick={() => setIsOpen(!isOpen)}>
        <span className="bar"></span>
        <span className="bar"></span>
        <span className="bar"></span>
      </div>
    </nav>
  );
};

export default Navbar;
```



PASO 3: Los Estilos y la Responsividad (Navbar.css)

Para lograr un aspecto visual dark, abre **Navbar.css** y pega el siguiente código.

```
/* =====  
ESTILOS GENERALES DEL NAVBAR  
===== */  
.navbar {  
  display: flex;  
  justify-content: space-between;  
  align-items: center;  
  padding: 1.5rem 5%;  
  background-color: #0b0f19; /* Fondo oscuro elegante */  
  color: #ffffff;  
  position: sticky;  
  top: 0;  
  z-index: 1000;  
  box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);  
}  
  
/* =====  
ESTILOS DEL LOGO  
===== */  
.navbar-logo {  
  font-size: 1.8rem;  
  font-weight: 700;  
  cursor: pointer;  
  letter-spacing: 1px;  
}  
  
.text-cyan {  
  color: #00d8ff; /* Color de acento (Cyan) */  
}  
  
/* =====  
ESTILOS DE LOS ENLACES (DESKTOP)  
===== */  
.navbar-links {  
  display: flex;  
  gap: 2.5rem;  
}  
  
.navbar-links a {  
  color: #a0aab2;  
  text-decoration: none;  
  font-size: 1rem;
```



```

font-weight: 500;
transition: color 0.3s ease;
}

.navbar-links a:hover {
  color: #ffffff;
}

/* =====
   BOTÓN HAMBURGUESA (Oculto en PC)
   ===== */
.navbar-toggle {
  display: none;
  flex-direction: column;
  cursor: pointer;
  gap: 6px;
}

.bar {
  width: 25px;
  height: 3px;
  background-color: #ffffff;
  border-radius: 2px;
  transition: all 0.3s ease;
}

/* =====
   MEDIA QUERY: VERSIÓN MÓVIL Y TABLET
   ===== */
@media (max-width: 768px) {
  .navbar-toggle {
    display: flex; /* Mostramos el botón hamburguesa */
  }

  .navbar-links {
    display: none; /* Ocultamos los enlaces horizontales */
    flex-direction: column;
    position: absolute;
    top: 100%;
    left: 0;
    width: 100%;
    background-color: #0b0f19;
    padding: 2rem 0;
    text-align: center;
    gap: 2rem;
    box-shadow: 0 10px 10px rgba(0, 0, 0, 0.2);
  }

  /* Cuando el estado isOpen es true, se añade la clase "open" */

```



```
.navbar-links.open {  
  display: flex;  
}  
}
```

PASO 4: Ensamblar en App.jsx

Finalmente, vamos a limpiar el código base que trae Vite por defecto e importaremos nuestro nuevo componente.

Abre **src/App.jsx** y déjalo exactamente así:

```
import Navbar from './components/Navbar/Navbar';  
import './App.css';  
import './index.css';  
  
function App() {  
  return (  
    <>  
      <Navbar />  
  
      /* El resto de secciones irán aquí debajo.  
       Ejemplo: <Footer />  
       */  
    </>  
  )  
}  
  
export default App;
```

 **Un consejo rápido para la clase:** Eliminen todo el contenido del **archivo index.css** que viene por defecto en Vite y solo dejen un reseteo básico de márgenes para que la barra negra ocupe el 100% de la pantalla sin bordes blancos:

```
/* En index.css */  
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
  font-family: 'Poppins', sans-serif;  
}
```



Creación del Banner Principal

PASO 5: El Componente React (Banner.jsx)

Crea una carpeta Banner y un archivo llamado Banner.jsx en tu carpeta src/components.

 **Nota para la clase:** Buscar una foto tuya en formato cuadrado (ej. 400x400px) y la guarden en la **carpeta src/assets** de tu proyecto. En este ejemplo, usaré una imagen de demostración.

```
import './Banner.css';
import miFoto from '../assets/miFoto.png';

const Banner = () => {
  return (
    <section className="hero-section">
      <div className="hero-content">

        {/* 1. Etiqueta de Estado (Badge) */}
        <div className="status-badge">
          <span className="status-dot"></span>
          Disponible para nuevos proyectos
        </div>

        {/* 2. Foto de Perfil Circular */}
        <div className="profile-photo-container">
          {/* Reemplazar src con {miFoto} cuando importen su imagen local */}
          <img
            src={miFoto}
            alt="Foto de Perfil"
            className="profile-photo"
          />
        </div>

        {/* 3. Título Principal */}
        <h1 className="hero-title">
          Giancarlos <br />
          <span className="text-cyan">Barboza</span>
        </h1>

        {/* 4. Subtítulo (Rol) */}
        <h2 className="hero-subtitle">
          Consultor e Instructor BI - Software | MOS Excel Expert | Power
          BI Data Analyst | Ingeniero de Sistemas
        </h2>
      </div>
    </section>
  )
}
```



```
</h2>

{/* 5. Descripción */}
<p className="hero-description">
  Construyo sistemas inteligentes que automatizan procesos,
  analizan
  datos y transforman la información. También desarrollo
  aplicaciones web funcionales para resolver necesidades reales.
  Consultoría de Software y BI para empresas.
</p>

{/* 6. Botones de Acción */}
<div className="hero-buttons">
  <a href="#proyectos" className="btn btn-primary">Ver
  Proyectos</a>
  <a href="#contacto" className="btn btn-
  secondary">Contáctame</a>
</div>

</div>

{/* 7. Indicador de Scroll (Flecha animada hacia abajo) */}
<div className="scroll-indicator">
  ↓
</div>
</section>
);
};

export default Banner;
```



PASO 6: Los Estilos y el Efecto Circular (Banner.css)

Crea el archivo **Banner.css** dentro de la carpeta Banner. Aquí aplicaremos un fondo oscuro. El secreto para la foto perfecta es la combinación de width, height, border-radius: 50% y object-fit: cover.

```
/* =====  
CONTENEDOR PRINCIPAL (Ocupa toda la pantalla)  
===== */  
.hero-section {  
  min-height: calc(100vh - 80px); /* 100vh menos la altura aproximada del  
  navbar */  
  display: flex;  
  flex-direction: column;  
  justify-content: center;  
  align-items: center;  
  background: radial-gradient(circle at center, #111a2e 0%, #0b0f19 70%);  
  color: #ffffff;  
  padding: 2rem 5%;  
  position: relative;  
}  
  
.hero-content {  
  max-width: 800px;  
  display: flex;  
  flex-direction: column;  
  align-items: center;  
  text-align: center;  
}  
  
/* =====  
ETIQUETA DE ESTADO (BADGE)  
===== */  
.status-badge {  
  display: flex;  
  align-items: center;  
  gap: 8px;  
  background-color: rgba(255, 255, 255, 0.03);  
  border: 1px solid rgba(255, 255, 255, 0.1);  
  padding: 0.5rem 1rem;  
  border-radius: 20px;  
  font-size: 0.85rem;  
  color: #a0aab2;  
  margin-bottom: 2rem;  
}
```



```
.status-dot {
  width: 8px;
  height: 8px;
  background-color: #00d8ff; /* Cyan */
  border-radius: 50%;
  box-shadow: 0 0 8px #00d8ff; /* Efecto de brillo */
}

/* =====
   FOTO DE PERFIL CIRCULAR
   ===== */
.profile-photo-container {
  margin-bottom: 1.5rem;
}

.profile-photo {
  width: 160px;
  height: 160px;
  object-fit: cover; /* Evita que la foto se estire si no es cuadrada */
  border-radius: 50%; /* Hace que sea un círculo perfecto */
  border: 6px solid rgba(5, 163, 215, 0.8); /* Borde sutil cyan */
  box-shadow: 0 0 20px rgba(5, 163, 215, 0.8); /* Sombra brillante */
}

/* =====
   TIPOGRAFÍA (Títulos y Descripción)
   ===== */
.hero-title {
  font-size: 4rem;
  font-weight: 700;
  line-height: 1.1;
  margin-bottom: 1.5rem;
}

.text-cyan {
  color: #00d8ff;
}

.hero-subtitle {
  font-size: 1.1rem;
  color: #00d8ff;
  font-weight: 500;
  margin-bottom: 1.5rem;
}

.hero-description {
  font-size: 1.1rem;
  color: #8b95a1;
}
```



```

line-height: 1.6;
margin-bottom: 2.5rem;
max-width: 700px;
}

/* =====
   BOTONES
===== */

.hero-buttons {
  display: flex;
  gap: 1rem;
}

.btn {
  padding: 0.8rem 2rem;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 600;
  text-decoration: none;
  transition: all 0.3s ease;
  cursor: pointer;
}

.btn-primary {
  background-color: #00d8ff;
  color: #0b0f19;
  border: 2px solid #00d8ff;
}

.btn-primary:hover {
  background-color: transparent;
  color: #00d8ff;
}

.btn-secondary {
  background-color: transparent;
  color: #ffffff;
  border: 1px solid rgba(255, 255, 255, 0.15);
}

.btn-secondary:hover {
  background-color: rgba(255, 255, 255, 0.05);
}

/* =====
   ANIMACIÓN DE LA FLECHA (Opcional)
===== */

.scroll-indicator {
  position: absolute;

```



```
bottom: 2rem;
color: #8b95a1;
font-size: 1.5rem;
animation: bounce 2s infinite;
}

@keyframes bounce {
  0%, 20%, 50%, 80%, 100% { transform: translateY(0); }
  40% { transform: translateY(-10px); }
  60% { transform: translateY(-5px); }
}

/* =====
   RESPONSIVIDAD (Móviles)
   ===== */
@media (max-width: 768px) {
  .hero-title {
    font-size: 2.8rem;
  }

  .hero-subtitle {
    font-size: 0.95rem;
  }

  .hero-description {
    font-size: 1rem;
  }

  .profile-photo {
    width: 110px;
    height: 110px;
  }

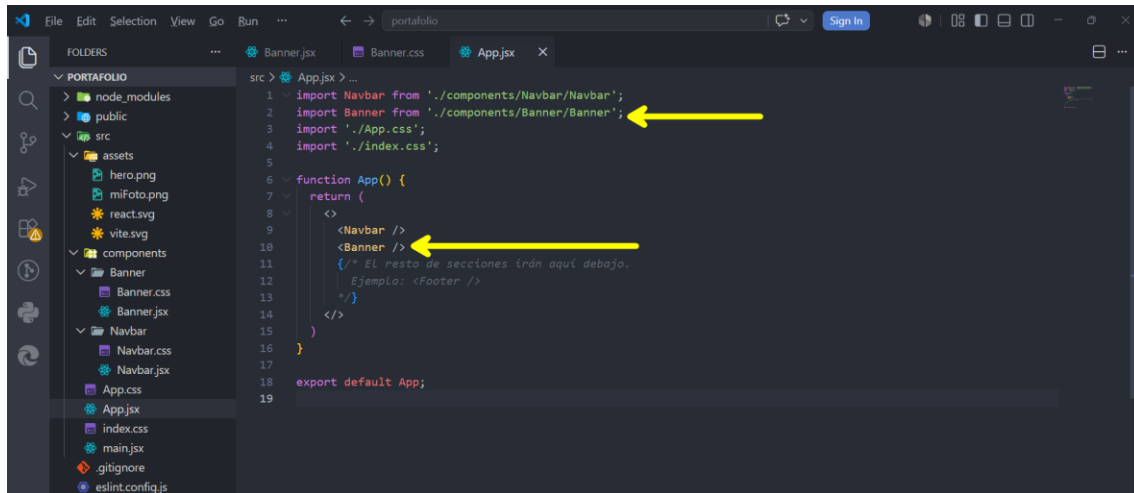
  /* En móviles, los botones se apilan uno debajo del otro */
  .hero-buttons {
    flex-direction: column;
    width: 100%;
  }

  .btn {
    width: 100%;
    text-align: center;
  }
}
```



PASO 7: Ensamblar en App.jsx

Para ver el resultado final, solo deben importar este nuevo componente en su **App.jsx**, justo debajo del **Navbar**.



```
1 import Navbar from '../components/Navbar/Navbar';
2 import Banner from '../components/Banner/Banner';
3 import './App.css';
4 import './index.css';
5
6 function App() {
7   return (
8     <>
9     <Navbar />
10    <Banner />
11    </* El resto de secciones irán aquí debajo.
12      Ejemplo: <Footer />
13    */ />
14   )
15 }
16
17 export default App;
```



PASO 8: El Componente React (Perfil.jsx)

En la carpeta **src/components**, crear una **carpeta de nombre Perfil** y dentro el archivo **Perfil.jsx**.

Nota didáctica: Para no depender de librerías externas de íconos (y mantener el proyecto ligero), vamos a usar SVGs integrados directamente en el código.

```
import './Perfil.css';

const Perfil = () => {
  return (
    <section id="sobre-mi" className="sobre-mi-section">
      <div className="sobre-mi-container">
        <h2 className="section-title">Sobre mí</h2>

        <div className="sobre-mi-grid">

          {/* =====
            TARJETA 1: PERFIL PROFESIONAL
            ===== */}

          <div className="card">
            <div className="card-header">
              {/* Ícono de Usuario (SVG) */}
              <div className="icon-box">
                <svg xmlns="http://www.w3.org/2000/svg" width="24"
height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
                  <path d="M20 21v-2a4 4 0 0 0-4-4H8a4 4 0 0 0-4
4v2"></path>
                  <circle cx="12" cy="7" r="4"></circle>
                </svg>
              </div>
              <h3>Perfil profesional</h3>
            </div>

            <p className="card-text">
              Ingeniero de Sistemas Computacionales, con experiencia en la
docencia de más de 15 años, dictando clases de forma presencial y virtual
en diferentes instituciones educativas de prestigio y con experiencia
profesional de más de 11 años.
              <br /><br />
              Cuento la Certificación Internacional en Microsoft Office
Specialist (MOS) Excel Expert y PL - 300 Data Analyst Associate.
Instructor CISCO y ORACLE.
            </p>
          </div>
        </div>
      </div>
    </section>
  )
}
```



```
<br /><br />
```

Consultoría y experiencia en desarrollo de software, utilizando tecnologías como React, Laravel, NodeJS, ExpressJS, MySQL, PostgreSQL, etc. y en desarrollo de dashboards interactivos mediante Power BI, DAX, Power Query, modelado de datos en SQL y automatizaciones con Python.

```
</p>
```

```
</div>
```

```
{/* =====
```

```
TARJETA 2: EDUCACIÓN
```

```
===== */}
```

```
<div className="card">
```

```
<div className="card-header">
```

```
{/* Ícono de Educación / Sombrero (SVG) */}
```

```
<div className="icon-box">
```

```
<svg xmlns="http://www.w3.org/2000/svg" width="24"
height="24" viewBox="0 0 24 24" fill="none" stroke="currentColor"
strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
```

```
<path d="M22 10v6M2 10l10-5 10 5-10 5z"></path>
```

```
<path d="M6 12v5c3 3 9 3 12 0v-5"></path>
```

```
</svg>
```

```
</div>
```

```
<h3>Educación</h3>
```

```
</div>
```

```
<div className="timeline">
```

```
{/* Ítem de Educación 1 */}
```

```
<div className="timeline-item">
```

```
<div className="timeline-header">
```

```
<h4>Ingeniería de Sistemas Computacionales</h4>
```

```
<span className="badge badge-outline">Titulado</span>
```

```
</div>
```

```
<span className="timeline-date">UNIVERSIDAD TECNOLÓGICA
DEL PERÚ</span>
```

```
</div>
```

```
{/* Ítem de Educación 2 */}
```

```
<div className="timeline-item">
```

```
<div className="timeline-header">
```

```
<h4>Computación e Informática</h4>
```

```
<span className="badge badge-outline">Titulado</span>
```

```
</div>
```

```
<span className="timeline-date">I.S.T CESCA</span>
```

```
</div>
```

```
{/* Ítem de Educación 3 */}
```

```
<div className="timeline-item">
```

```
<div className="timeline-header">
```



```

        <h4>MOS Excel Expert</h4>
        <span className="badge badge-outline">Certificación
Internacional</span>
      </div>
      <span className="timeline-date">MICROSOFT</span>
    </div>

    { /* Ítem de Educación 4 */ }
    <div className="timeline-item">
      <div className="timeline-header">
        <h4>Power BI Data Analyst Associate - PL - 300</h4>
        <span className="badge badge-outline">Certificación
Internacional</span>
      </div>
      <span className="timeline-date">MICROSOFT</span>
    </div>
  </div>
</div>
</section>
);
};

export default Perfil;

```

PASO 9: Los Estilos (Perfil.css)

Crea el archivo Perfil.css. Usaremos **CSS Grid**, una herramienta poderosa para maquetar estas dos tarjetas una al lado de la otra, dándole a la del perfil un poco más de ancho.

```

/* =====
CONTENEDOR PRINCIPAL
===== */

.sobre-mi-section {
  padding: 5rem 5%;
  background-color: #0b0f19; /* Continuidad del fondo oscuro */
  color: #ffffff;
  display: flex;
  justify-content: center;
}

.sobre-mi-container {
  max-width: 1000px;
  width: 100%;
}

```



```
.section-title {
  font-size: 2.5rem;
  font-weight: 700;
  margin-bottom: 2.5rem;
  color: #ffffff;
}

/* =====
   GRID (Grilla para las 2 tarjetas)
   ===== */
.sobre-mi-grid {
  display: grid;
  /* La primera columna será ligeramente más ancha que la segunda (1.3fr vs 1fr) */
  grid-template-columns: 1.3fr 1fr;
  gap: 2rem;
}

/* =====
   TARJETAS (Cards)
   ===== */
.card {
  background-color: #111a2e; /* Un tono ligeramente más claro que el fondo */
  border: 1px solid rgba(255, 255, 255, 0.05);
  border-radius: 12px;
  padding: 2.5rem;
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
}

.card-header {
  display: flex;
  align-items: center;
  gap: 1rem;
  margin-bottom: 1.5rem;
}

.icon-box {
  background-color: rgba(110, 231, 183, 0.1);
  color: #6ee7b7; /* Color menta/cian */
  padding: 0.8rem;
  border-radius: 8px;
  display: flex;
  align-items: center;
  justify-content: center;
}

.card-header h3 {
```



```
font-size: 1.4rem;
font-weight: 600;
color: #ffffff;
}

.card-text {
  color: #8b95a1;
  line-height: 1.8;
  font-size: 1rem;
  text-align: justify;
}

/* =====
   LÍNEA DE TIEMPO (Educación)
   ===== */

.timeline {
  display: flex;
  flex-direction: column;
  gap: 2rem;
}

.timeline-item {
  position: relative;
  padding-left: 1.5rem;
  border-left: 2px solid rgba(255, 255, 255, 0.1);
}

/* El punto en la línea de tiempo */
.timeline-item::before {
  content: '';
  position: absolute;
  left: -5px; /* Centra el punto sobre el borde izquierdo */
  top: 5px;
  width: 8px;
  height: 8px;
  background-color: #111a2e;
  border: 2px solid #6ee7b7;
  border-radius: 50%;
}

.timeline-header {
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  margin-bottom: 0.5rem;
  flex-wrap: wrap;
  gap: 0.5rem;
}
```



```
.timeline-header h4 {
  font-size: 1.1rem;
  color: #6ee7b7;
  font-weight: 600;
}

.timeline-date {
  font-size: 0.9rem;
  color: #8b95a1;
}

/* =====
   ETIQUETAS (Badges)
   ===== */
.badge {
  font-size: 0.75rem;
  font-weight: 700;
  padding: 0.2rem 0.6rem;
  border-radius: 4px;
  letter-spacing: 0.5px;
}

.badge-outline {
  border: 1px solid #6ee7b7;
  color: #6ee7b7;
  background-color: transparent;
}

.badge-solid {
  background-color: #6ee7b7;
  color: #0b0f19;
}

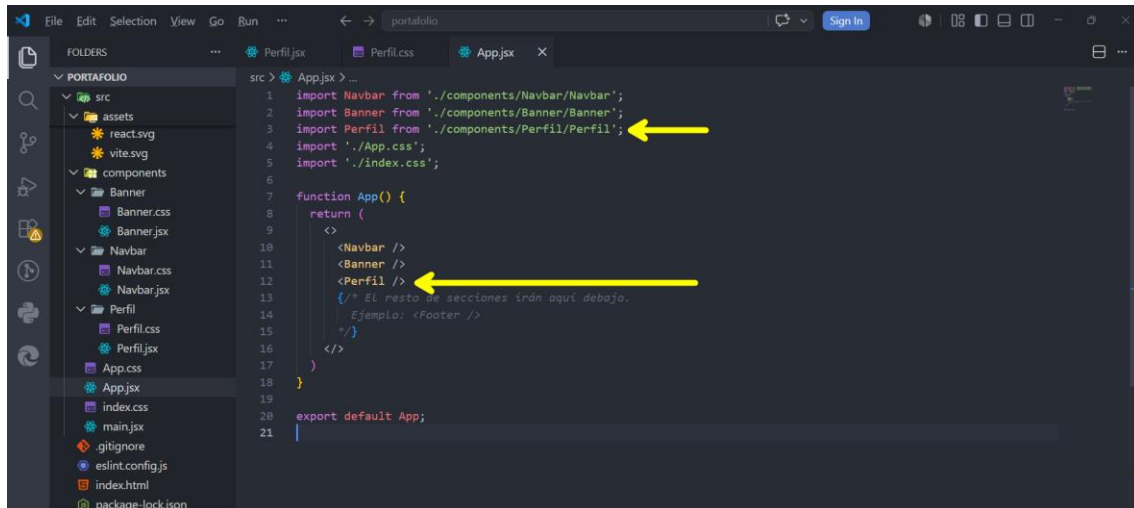
/* =====
   RESPONSABILIDAD (Celulares y Tablets)
   ===== */
@media (max-width: 900px) {
  .sobre-mi-grid {
    /* En pantallas pequeñas, colapsa a 1 sola columna */
    grid-template-columns: 1fr;
  }

  .card {
    padding: 1.5rem; /* Reduce el padding interno en móviles */
  }
}
```



PASO 10: Ensamblar en App.jsx

Ya sabemos la rutina. Vamos al archivo raíz e importamos la nueva sección debajo del Banner.



```

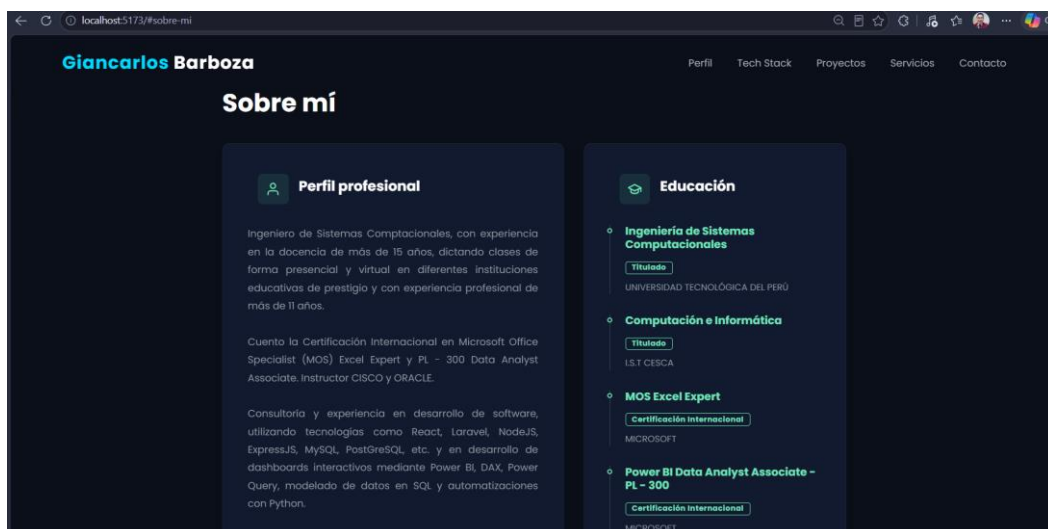
1 import Navbar from '../components/Navbar/Navbar';
2 import Banner from '../components/Banner/Banner';
3 import Perfil from '../components/Perfil/Perfil';
4 import './App.css';
5 import './index.css';
6
7 function App() {
8   return (
9     <>
10      <Navbar />
11      <Banner />
12      <Perfil />
13      { /* El resto de secciones irán aquí debajo.
14        Ejemplo: <Footer />
15      */ }
16    </>
17  );
18 }
19
20 export default App;
21

```

💡 Momento didáctico para la clase:

Hacemos una pausa. ¿Qué pasaría si no tuvieramos la media query `@media (max-width: 900px)`?

Esto te permitirá demostrarles que, al achicar la pantalla, las dos columnas se aplastarían volviendo el texto ilegible. Explicarles cómo la media query cambia el grid-template-columns de `1.3fr 1fr` a simplemente `1fr` es el mejor ejemplo práctico para que entiendan la lógica del diseño Responsive.



PASO 11: El Componente React (Habilidades.jsx)

Crea la **carpeta Habilidades** y dentro el archivo **Habilidades.jsx** en la carpeta **src/components**.

 **Nota para la clase:** En este código se ha colocado un **arreglo** llamado **skillsData**. Para separar los datos de la interfaz gráfica es una de las mejores prácticas en el desarrollo de software. Para los íconos, se ha puesto un SVG genérico de ejemplo, pero pueden cambiar la etiqueta `<svg>` por un `<img src={...}/>` si ya descargaron los logos oficiales.

```
import './Habilidades.css';

const Habilidades = () => {
  // Arreglo de datos: Simulamos la información que podría venir de una API
  const skillsData = [
    {
      id: 1,
      titulo: "Power BI",
      descripcion: "DAX, Data Modeling, Power Query, Governance.",
      nivel: "Avanzado",
      botonTexto: "VER CERTIFICADO",
      estado: "completado" // Usaremos esto para cambiar los colores
    },
    {
      id: 2,
      titulo: "SQL Server / MySQL / PostgreSQL / Oracle",
      descripcion: "SQL, T-SQL, Optimización, Procedimientos Almacenados.",
      nivel: "Avanzado",
      botonTexto: "VER CERTIFICADO",
      estado: "completado"
    },
    {
      id: 3,
      titulo: "Python for Data Analyst",
      descripcion: "Pandas, NumPy, Matplotlib, Automatización, Keras.",
      nivel: "Avanzado",
      botonTexto: "VER CERTIFICADO",
      estado: "completado"
    },
    {
      id: 4,
      titulo: "Excel / VBA",
```



```
    descripcion: "Macros, Power Pivot, Power Query, Pivot Tables,
Dashboards complejos.",
    nivel: "Experto",
    botonTexto: "VER CERTIFICADO",
    estado: "completado"
  },
  {
    id: 5,
    titulo: "Laravel",
    descripcion: "Backend Developer",
    nivel: "Avanzado",
    botonTexto: "VER CERTIFICADO",
    estado: "completado"
  },
  {
    id: 6,
    titulo: "React",
    descripcion: "Construcción de Interfaces de Usuario Modernas a
partir de Componentes",
    nivel: "Aprendizaje Activo",
    botonTexto: "VER CERTIFICADO",
    estado: "completado"
  },
  {
    id: 7,
    titulo: "Angular",
    descripcion: "Construcción de aplicaciones web de una sola
página.",
    nivel: "Aprendizaje Activo",
    botonTexto: "VER CERTIFICADO",
    estado: "completado"
  },
  {
    id: 8,
    titulo: "Git",
    descripcion: "Sistema de Control de Versiones.",
    nivel: "Aprendizaje Activo",
    botonTexto: "VER CERTIFICADO",
    estado: "completado"
  },
  {
    id: 9,
    titulo: "GitHub",
    descripcion: "Repositorio en la nube.",
    nivel: "Aprendizaje Activo",
    botonTexto: "VER CERTIFICADO",
    estado: "completado"
  },
  {
```



```

        id: 10,
        titulo: "JavaScript",
        descripcion: "Lenguaje de programación interpretado, dialecto del
estándar ECMAScript.",
        nivel: "Aprendizaje Activo",
        botonTexto: "VER CERTIFICADO",
        estado: "completado"
    },
    {
        id: 11,
        titulo: "Grid",
        descripcion: "Contrucción de rejillas responsivas.",
        nivel: "Aprendizaje Activo",
        botonTexto: "VER CERTIFICADO",
        estado: "completado"
    },
    {
        id: 12,
        titulo: "Flexbox",
        descripcion: "Método de diseño de página unidimensional para
compaginar elementos en filas o columnas.",
        nivel: "Aprendizaje Activo",
        botonTexto: "VER CERTIFICADO",
        estado: "completado"
    }
];

return (
    <section id="tech-stack" className="habilidades-section">
        <div className="habilidades-container">
            /* Iteramos sobre el arreglo de datos usando .map() */
            <div className="habilidades-grid">
                {skillsData.map((skill) => (
                    <div key={skill.id} className="skill-card">

                        /* Cabecera de la tarjeta: Ícono y Botón */
                        <div className="skill-header">
                            <div className="skill-icon">
                                /* Aquí pueden colocar la etiqueta <img
src={LogoPowerBI} /> */
                                <svg width="32" height="32" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round" className="text-cyan">
                                    <rect x="3" y="3" width="18" height="18" rx="2"
ry="2"></rect>

                                    <line x1="3" y1="9" x2="21" y2="9"></line>
                                    <line x1="9" y1="21" x2="9" y2="9"></line>
                                </svg>

```



```

        </div>

        { /* Clases dinámicas: Si el estado es proceso, aplicamos
        la clase 'btn-proceso' */}
        <span className={`skill-badge ${skill.estado ===
        'proceso' ? 'badge-yellow' : 'badge-green'}`}>
            {skill.botonTexto}
        </span>
    </div>

    { /* Cuerpo de La tarjeta: Textos */}
    <div className="skill-body">
        <h3 className="skill-title">{skill.titulo}</h3>
        <p className="skill-desc">{skill.descripcion}</p>
        <span className="skill-level">{skill.nivel}</span>
    </div>

    </div>
    )}
</div>
</div>
</section>
);
};

export default Habilidades;

```

PASO 12: Los Estilos y la Cuadrícula Automática (Habilidades.css)

Crea el archivo **Habilidades.css**. Aquí usaremos un truco de **CSS Grid** llamado **auto-fit con minmax()**. Esto es magia pura para el diseño responsive: le dice al navegador que meta tantas columnas como quepan, siempre y cuando ninguna sea más pequeña de 300px. Así te ahorras tener que escribir múltiples Media Queries.

```

/* =====
CONTENEDOR PRINCIPAL
===== */
.habilidades-section {
    padding: 5rem 5%;
    background-color: #0b0f19;
    display: flex;
    justify-content: center;
}

.habilidades-container {

```



```

    max-width: 1200px; /* Un poco más ancho para albergar 3 columnas
    cómodamente */
    width: 100%;
}

/* =====
   GRID AUTOMÁTICO (Magia Responsive)
   ===== */
.habilidades-grid {
    display: grid;
    /* Crea columnas automáticamente que no sean menores a 320px */
    grid-template-columns: repeat(auto-fit, minmax(320px, 1fr));
    gap: 1.5rem;
}

/* =====
   TARJETA (Card)
   ===== */
.skill-card {
    background-color: #111a2e;
    border: 1px solid rgba(255, 255, 255, 0.05);
    border-radius: 12px;
    padding: 2rem;
    transition: transform 0.3s ease, box-shadow 0.3s ease;
    display: flex;
    flex-direction: column;
    justify-content: space-between;
}

/* Efecto al pasar el mouse */
.skill-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 10px 25px rgba(0, 0, 0, 0.3);
    border-color: rgba(110, 231, 183, 0.2);
}

/* =====
   CABECERA (Ícono y Botón)
   ===== */
.skill-header {
    display: flex;
    justify-content: space-between;
    align-items: flex-start;
    margin-bottom: 1.5rem;
}

.skill-icon {
    width: 48px;
    height: 48px;

```



```

display: flex;
align-items: center;
justify-content: flex-start;
}

.text-cyan {
  color: #00d8ff;
}

/* =====
   BOTONES / ETIQUETAS SUPERIORES
===== */

.skill-badge {
  font-size: 0.75rem;
  font-weight: 700;
  padding: 0.4rem 0.8rem;
  border-radius: 20px;
  letter-spacing: 0.5px;
  border: 1px solid;
  background-color: transparent;
  cursor: pointer;
  transition: background-color 0.3s ease;
}

/* Variante Verde (Certificado) */
.badge-green {
  color: #6ee7b7;
  border-color: rgba(110, 231, 183, 0.4);
}

.badge-green:hover {
  background-color: rgba(110, 231, 183, 0.1);
}

/* Variante Amarilla (En Proceso) */
.badge-yellow {
  color: #fbbf24;
  border-color: rgba(251, 191, 36, 0.4);
}

.badge-yellow:hover {
  background-color: rgba(251, 191, 36, 0.1);
}

/* =====
   CUERPO DE LA TARJETA (Textos)
===== */

.skill-title {
  color: #ffffff;

```



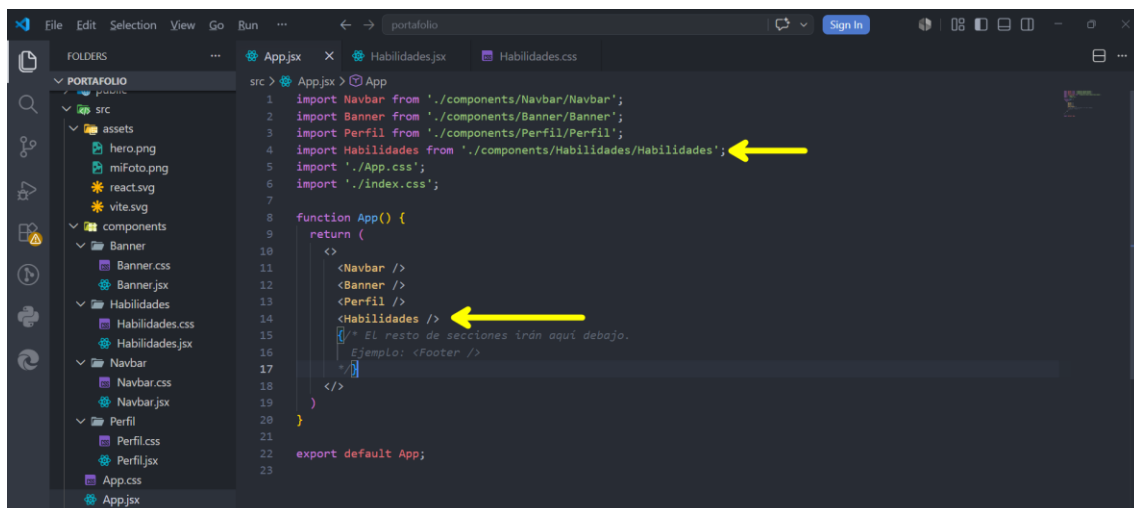
```
font-size: 1.25rem;
font-weight: 700;
margin-bottom: 0.8rem;
}

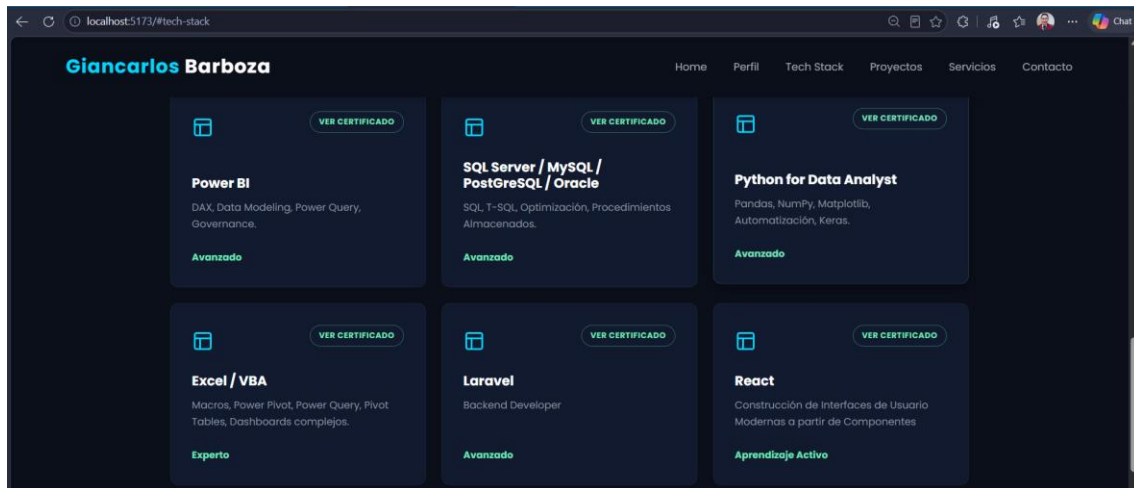
.skill-desc {
  color: #8b95a1;
  font-size: 0.95rem;
  line-height: 1.6;
  margin-bottom: 1.5rem;
  min-height: 50px; /* Mantiene la altura pareja aunque el texto sea
corto */
}

.skill-level {
  color: #6ee7b7;
  font-size: 0.9rem;
  font-weight: 600;
}
```

PASO 13: Ensamblar en App.jsx

Finalmente, volvemos al lienzo principal para incrustar esta nueva sección debajo de la sección de perfil.





PASO 14: El Componente React (Proyectos.jsx)

Crear el archivo **Proyectos.jsx** en la carpeta **src/components/Proyectos**.

Nota para la clase: Para las imágenes, se usará enlaces públicos de algunos de mis proyectos realizados. Tomar capturas de pantalla de sus propios proyectos, guardarlas en la carpeta **assets** e importarlas tal como hicieron con su foto de perfil.

```
import './Proyectos.css';

const Proyectos = () => {
  // Arreglo de proyectos (Simulando una base de datos o API)
  const proyectosData = [
    {
      id: 1,
      imagen: "https://foria.com.pe/imagenes/zap2356.jpg",
      titulo: "Desarrollo de Aplicación Web y Carrito de Compras - FORIA SAC",
      descripcion: "Aplicación web de venta de calzados para varones, mujeres, niños y niñas.",
      tecnologias: ["HTML5", "CSS3", "JavaScript", "React", "Laravel", "PostgreSQL"],
      linkDemo: "https://foria.com.pe",
      linkCodigo: "#"
    },
    {
      id: 2,
      imagen: "https://fravia.com.pe/imagenes/HOMBRE-cAT.jpg",
      titulo: "Desarrollo de Aplicación Web y Carrito de Compras - FRAVIA SAC",
    }
  ]
}
```



```

        descripcion: "Aplicación web de venta de ropa para varones,
mujeres, niños y niñas.",
        tecnologias: ["HTML5", "CSS3", "JavaScript", "React", "Laravel",
"PostgreSQL"],
        linkDemo: "https://fravia.com.pe",
        linkCodigo: "#"
    },
    {
        id: 3,
        imagen: "https://intikisa.pe/wp-
content/uploads/2024/08/Productos_en_polvo1.png.webp",
        titulo: "Aplicación Web de venta de productos naturales",
        descripcion: "Aplicación web para la venta de productos naturales
online.",
        tecnologias: ["React", "Laravel", "PostgreSQL"],
        linkDemo: "https://intikisa.pe",
        linkCodigo: "#"
    },
    {
        id: 4,
        imagen: "https://liora.io/app/uploads/sites/7/2020/10/power-bi-
logo-1.jpg",
        titulo: "Dashboard Interactivo en Power BI",
        descripcion: "Construcción de un Panel BI Interactivo.",
        tecnologias: ["Power BI", "DAX", "Power Query", "Modelado de
datos", "Power BI Service", "SharePoint"],
        linkDemo: "#",
        linkCodigo: "#"
    }
];

return (
    <section id="proyectos" className="proyectos-section">
        <div className="proyectos-container">
            <h2 className="section-title text-center">Proyectos
Destacados</h2>

            <div className="proyectos-grid">
                {proyectosData.map((proyecto) => (
                    <article key={proyecto.id} className="proyecto-card">

                        {/* 1. Imagen del Proyecto */}
                        <div className="proyecto-imagen">
                            <img src={proyecto.imagen} alt={`Captura de
${proyecto.titulo}`} />
                        </div>

                        {/* 2. Contenido de La Tarjeta */}
                        <div className="proyecto-contenido">

```



```

<h3 className="proyecto-titulo">{proyecto.titulo}</h3>
<p className="proyecto-desc">{proyecto.descripcion}</p>

{/* 3. Etiquetas de Tecnologías (Mapeo Anidado) */}
<div className="proyecto-tags">
  {proyecto.tecnologias.map((tech, index) => (
    <span key={index} className="tech-tag">
      {tech}
    </span>
  ))}
</div>

{/* 4. Botones de Acción */}
<div className="proyecto-links">
  <a href={proyecto.linkDemo} target="_blank"
rel="noopener noreferrer" className="btn-proyecto btn-demo">
    {/* Ícono de Enlace Externo */}
    <svg width="18" height="18" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round">
      <path d="M18 13v6a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2V8a2
2 0 0 1 2-2h6"></path>
      <polyline points="15 3 21 3 21 9"></polyline>
      <line x1="10" y1="14" x2="21" y2="3"></line>
    </svg>
    Demo en Vivo
  </a>

  <a href={proyecto.linkCodigo} target="_blank"
rel="noopener noreferrer" className="btn-proyecto btn-codigo">
    {/* Ícono de GitHub */}
    <svg width="18" height="18" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round">
      <path d="M9 19c-5 1.5-5-2.5-7-3m14 6v-3.87a3.37
3.37 0 0 0-.94-2.61c3.14-.35 6.44-1.54 6.44-7A5.44 5.44 0 0 0 20 4.77
5.07 5.07 0 0 0 19.91 1S18.73.65 16 2.48a13.38 13.38 0 0 0-7 0C6.27.65
5.09 1 5.09 1A5.07 5.07 0 0 0 5 4.77a5.44 5.44 0 0 0-1.5 3.78c0 5.42 3.3
6.61 6.44 7A3.37 3.37 0 0 0 9 18.13V22"></path>
    </svg>
    Ver Código
  </a>
</div>
</div>

</article>
  )})
</div>
</div>

```



```

    </section>
  );
};

export default Proyectos;

```

PASO 15: Los Estilos (Proyectos.css)

Crea el archivo **Proyectos.css**. Aquí el truco principal es que la imagen no tenga bordes redondeados en la parte inferior, y que el contenedor del contenido use Flexbox (flex-grow: 1) para empujar los botones siempre al fondo de la tarjeta, incluso si un proyecto tiene más texto que otro.

```

/* =====
CONTENEDOR PRINCIPAL
===== */
.proyectos-section {
  padding: 5rem 5%;
  background-color: #0b0f19;
  display: flex;
  justify-content: center;
}

.proyectos-container {
  max-width: 1200px;
  width: 100%;
}

.text-center {
  text-align: center;
}

/* =====
GRID (2 Columnas en PC)
===== */
.proyectos-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(450px, 1fr));
  gap: 2.5rem;
}

/* =====
TARJETA DEL PROYECTO
===== */
.proyecto-card {
  background-color: #111a2e;

```



```
border: 1px solid rgba(255, 255, 255, 0.05);
border-radius: 16px;
overflow: hidden; /* Clave para que la imagen no se salga de las curvas */
display: flex;
flex-direction: column;
box-shadow: 0 4px 20px rgba(0, 0, 0, 0.2);
transition: transform 0.3s ease;
}

.proyecto-card:hover {
  transform: translateY(-8px);
}

/* =====
   IMAGEN DEL PROYECTO
   ===== */
.proyecto-imagen img {
  width: 100%;
  height: 280px;
  object-fit: cover; /* Evita que la captura se deforme */
  border-bottom: 1px solid rgba(255, 255, 255, 0.05);
}

/* =====
   CONTENIDO Y TEXTOS
   ===== */
.proyecto-contenido {
  padding: 2rem;
  display: flex;
  flex-direction: column;
  flex-grow: 1; /* Empuja el resto hacia abajo */
}

.proyecto-titulo {
  color: #ffffff;
  font-size: 1.5rem;
  font-weight: 700;
  margin-bottom: 1rem;
}

.proyecto-desc {
  color: #8b95a1;
  line-height: 1.6;
  font-size: 1rem;
  margin-bottom: 1.5rem;
  flex-grow: 1; /* Permite que este párrafo crezca si es necesario */
}
```



```
/* =====  
ETIQUETAS (Tags de Tecnologías)  
===== */  
  
.proyecto-tags {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 0.8rem;  
  margin-bottom: 2rem;  
}  
  
.tech-tag {  
  background-color: rgba(255, 255, 255, 0.05);  
  border: 1px solid rgba(255, 255, 255, 0.1);  
  color: #a0aab2;  
  padding: 0.4rem 0.8rem;  
  border-radius: 20px;  
  font-size: 0.85rem;  
  font-weight: 500;  
}  
  
/* =====  
BOTONES DE ACCIÓN  
===== */  
  
.proyecto-links {  
  display: flex;  
  gap: 1rem;  
  flex-wrap: wrap;  
}  
  
.btn-proyecto {  
  display: flex;  
  align-items: center;  
  gap: 0.5rem;  
  padding: 0.7rem 1.5rem;  
  border-radius: 8px;  
  font-size: 0.95rem;  
  font-weight: 600;  
  text-decoration: none;  
  transition: all 0.3s ease;  
}  
  
.btn-demo {  
  color: #00d8ff;  
  border: 1px solid #00d8ff;  
  background-color: rgba(0, 216, 255, 0.05);  
}  
  
.btn-demo:hover {  
  background-color: rgba(0, 216, 255, 0.15);
```



```

}

.btn-codigo {
  color: #ffffff;
  border: 1px solid rgba(255, 255, 255, 0.2);
  background-color: transparent;
}

.btn-codigo:hover {
  background-color: rgba(255, 255, 255, 0.1);
}

/* =====
   RESPONSIVIDAD (Móviles)
   ===== */
@media (max-width: 600px) {
  .proyectos-grid {
    grid-template-columns: 1fr; /* 1 sola columna forzada */
  }

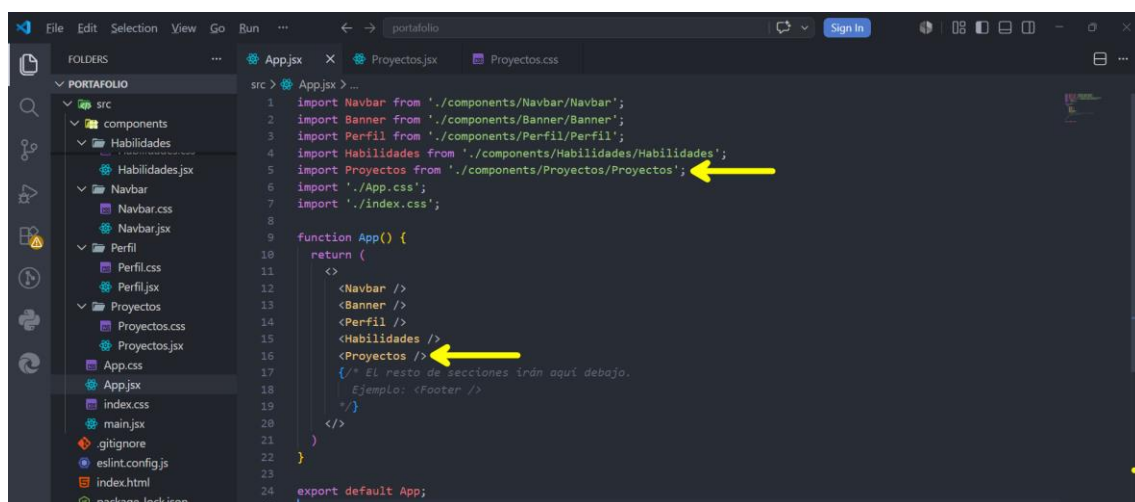
  .proyecto-links {
    flex-direction: column;
  }

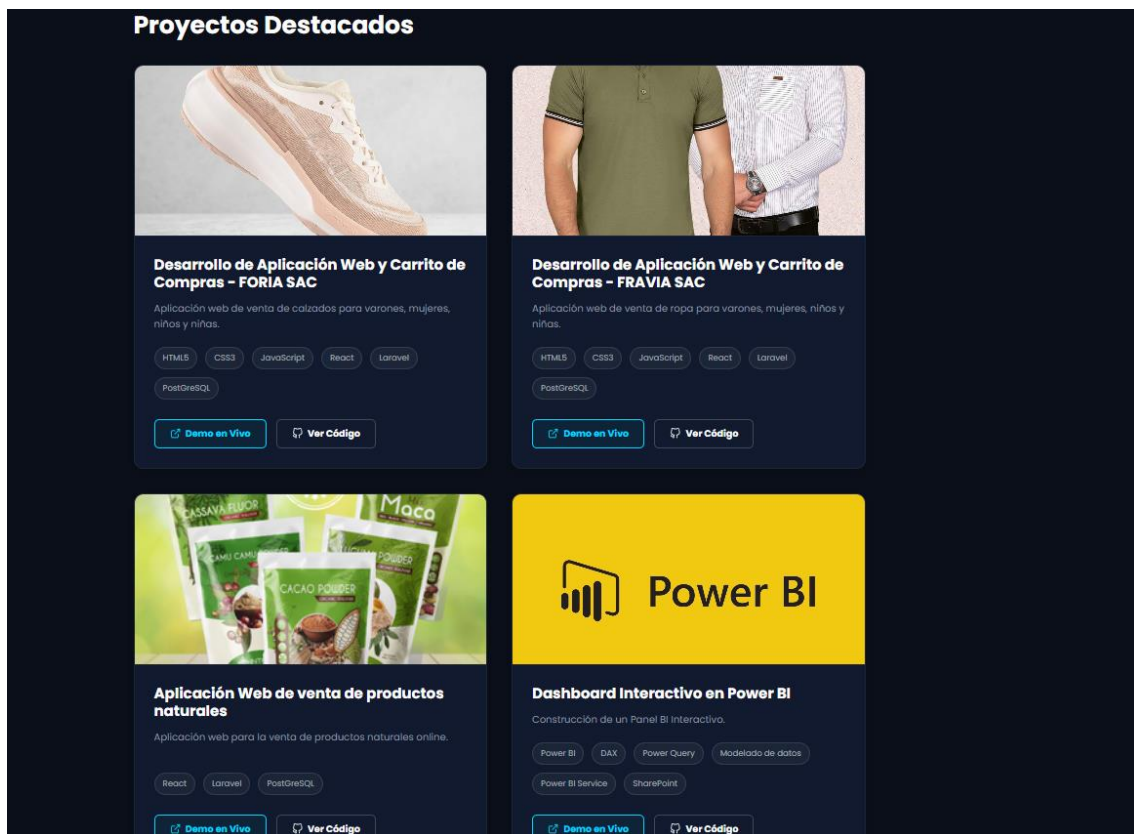
  .btn-proyecto {
    justify-content: center;
  }
}

```

PASO 16: Ensamblar en App.jsx

Por último, los alumnos solo deben importar este componente en el archivo principal para añadir la nueva capa a su sitio.





PASO 17: El Componente React (Contacto.jsx)

Crear el archivo **Contacto.jsx** en la carpeta **src/components/Contacto**.

Implementaremos un estado para capturar lo que el usuario escribe y una función para simular el envío.

```
import { useState } from 'react';
import './Contacto.css';

const Contacto = () => {
  // Estado centralizado para el formulario (Formulario Controlado)
  const [formulario, setFormulario] = useState({
    nombre: '',
    email: '',
    mensaje: ''
  });

  // Función para manejar los cambios en los inputs
  const manejarCambio = (e) => {
    const { name, value } = e.target;
    setFormulario({
      ...formulario,
```



```

    [name]: value
  });
};

// Función para simular el envío
const manejarEnvio = (e) => {
  e.preventDefault();
  alert(`¡Gracias por tu mensaje, ${formulario.nombre}! Te contactaré pronto.`);
  // Limpiamos el formulario después de enviar
  setFormulario({ nombre: '', email: '', mensaje: '' });
};

return (
  <section id="contacto" className="contacto-section">
    <div className="contacto-container">
      <h2 className="section-title">Trabajemos Juntos</h2>
      <div className="contacto-grid">

        {/* =====
           COLUMNA IZQUIERDA: INFORMACIÓN
           ===== */}
        <div className="contacto-info">
          <h3 className="info-title">Ponte en Contacto</h3>
          <p className="info-desc">
            ¿Tienes un proyecto en mente o necesitas ayuda para
            automatizar tus flujos de trabajo?
            Me encantaría escucharte.
          </p>

          <div className="info-items">
            {/* Ítem Email */}
            <div className="info-item">
              <div className="info-icon">
                <svg width="20" height="20" viewBox="0 0 24 24"
                fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
                strokeLinejoin="round">
                  <path d="M4 4h16c1.1 0 2 .9 2 2v12c0 1.1-.9 2-2 2H4c-
1.1 0-2-.9-2-2V6c0-1.1.9-2 2-2z"></path>
                  <polyline points="22,6 12,13 2,6"></polyline>
                </svg>
              </div>
              <div>
                <span className="item-label">Email</span>
                <p className="item-text">gbarboza.es@gmail.com</p>
              </div>
            </div>

            {/* Ítem Ubicación */}

```



```

        <div className="info-item">
            <div className="info-icon">
                <svg width="20" height="20" viewBox="0 0 24 24"
fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round"
strokeLinejoin="round">
                    <path d="M21 10c0 7-9 13-9 13s-9-6-9-13a9 9 0 0 1 18
0z"></path>
                    <circle cx="12" cy="10" r="3"></circle>
                </svg>
            </div>
            <div>
                <span className="item-label">Ubicación</span>
                <p className="item-text"> Lima - Perú</p>
            </div>
        </div>
    </div>
</div>

{ /* =====
    COLUMNA DERECHA: FORMULARIO
    ===== */ }
<div className="contacto-form-wrapper">
    <form onSubmit={manejarEnvio} className="contacto-form">

        <div className="form-group">
            <label htmlFor="nombre">Nombre</label>
            <input
                type="text"
                id="nombre"
                name="nombre"
                value={formulario.nombre}
                onChange={manejarCambio}
                placeholder="Tu nombre"
                required
            />
        </div>

        <div className="form-group">
            <label htmlFor="email">Email</label>
            <input
                type="email"
                id="email"
                name="email"
                value={formulario.email}
                onChange={manejarCambio}
                placeholder="tu@email.com"
                required
            />
        </div>
    </form>
</div>

```



```

        <div className="form-group">
          <label htmlFor="mensaje">Mensaje</label>
          <textarea
            id="mensaje"
            name="mensaje"
            value={formulario.mensaje}
            onChange={manejarCambio}
            placeholder="Cuéntame sobre tu proyecto..."
            rows="5"
            required
          ></textarea>
        </div>

        <button type="submit" className="btn-submit">Enviar
Mensaje</button>
      </form>
    </div>

  </div>
</div>
</section>
);
};

export default Contacto;

```

PASO 18: Los Estilos (Contacto.css)

Crea el archivo **Contacto.css**. El enfoque principal aquí es estilizar los inputs y el textarea para que tengan ese fondo oscuro transparente y cambien el color del borde a cyan (#00d8ff) cuando el usuario hace clic en ellos (usando la pseudo-clase :focus).

```

/* =====
CONTENEDOR PRINCIPAL
===== */
.contacto-section {
  padding: 6rem 5%;
  background-color: #050810; /* Un fondo ligeramente más oscuro para
cerrar la página */
  color: #ffffff;
  display: flex;
  flex-direction: column;
  align-items: center;
}

```



```
.contacto-main-title {
  color: #00d8ff;
  font-size: 2.5rem;
  font-weight: 700;
  margin-bottom: 4rem;
  text-align: center;
}

.contacto-container {
  max-width: 1000px;
  width: 100%;
}

/* =====
   GRID (2 Columnas)
===== */
.contacto-grid {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 4rem;
  align-items: start;
}

/* =====
   COLUMNA IZQUIERDA (Info)
===== */
.info-title {
  font-size: 1.8rem;
  font-weight: 600;
  margin-bottom: 1rem;
}

.info-desc {
  color: #8b95a1;
  line-height: 1.6;
  margin-bottom: 2.5rem;
}

.info-items {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}

.info-item {
  display: flex;
  align-items: center;
  gap: 1rem;
}
```



```
.info-icon {
  background-color: rgba(255, 255, 255, 0.03);
  border: 1px solid rgba(255, 255, 255, 0.1);
  color: #00d8ff;
  width: 45px;
  height: 45px;
  border-radius: 12px;
  display: flex;
  align-items: center;
  justify-content: center;
}

.item-label {
  display: block;
  font-size: 0.85rem;
  color: #8b95a1;
  margin-bottom: 0.2rem;
}

.item-text {
  font-size: 1rem;
  font-weight: 500;
  color: #ffffff;
}

/* =====
   COLUMNA DERECHA (Formulario)
   ===== */

.contacto-form-wrapper {
  background-color: #111a2e;
  padding: 2.5rem;
  border-radius: 16px;
  border: 1px solid rgba(255, 255, 255, 0.05);
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
}

.contacto-form {
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}

.form-group {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}
```



```
.form-group label {
  font-size: 0.9rem;
  color: #a0aab2;
}

/* Estilos para los campos de entrada */
.form-group input,
.form-group textarea {
  width: 100%;
  background-color: rgba(0, 0, 0, 0.2);
  border: 1px solid rgba(255, 255, 255, 0.1);
  color: #ffffff;
  padding: 0.8rem 1rem;
  border-radius: 8px;
  font-size: 1rem;
  font-family: inherit;
  outline: none;
  transition: all 0.3s ease;
}

/* Efecto FOCUS (Borde Cyan al hacer clic) */
.form-group input:focus,
.form-group textarea:focus {
  border-color: #00d8ff;
  box-shadow: 0 0 0 2px rgba(0, 216, 255, 0.2);
}

.form-group input::placeholder,
.form-group textarea::placeholder {
  color: #4b5563;
}

.form-group textarea {
  resize: vertical;
}

/* =====
   BOTÓN ENVIAR
   ===== */
.btn-submit {
  background-color: #00d8ff;
  color: #050810;
  border: none;
  padding: 1rem;
  border-radius: 8px;
  font-size: 1rem;
  font-weight: 700;
  cursor: pointer;
  transition: background-color 0.3s ease, transform 0.2s ease;
}
```



```

margin-top: 0.5rem;
}

.btn-submit:hover {
  background-color: #00bce6;
  transform: translateY(-2px);
}

/* =====
   RESPONSIVIDAD (Móviles)
   ===== */
@media (max-width: 768px) {
  .contacto-grid {
    grid-template-columns: 1fr;
    gap: 3rem;
  }

  .contacto-main-title {
    font-size: 2rem;
    margin-bottom: 2rem;
  }

  .contacto-form-wrapper {
    padding: 1.5rem;
  }
}

```

PASO 19: Ensamblar en App.jsx

Solo queda importar este último componente en el archivo principal y el portafolio de tus estudiantes estará completo.

